Start here: SOC, SRE and this project

A one-page primer for readers without a security or engineering background

Project description. Security tools raise thousands of alerts a day, and most are never read. We replay real Suricata alerts from the CIC-IDS2017 Thursday data through two SIEM mocks. SIEM I scores every alert on its own. Codex then acts like an SRE for the alert stream: it measures the flow per rule, compares it with an analyst budget, and tunes the grouping window and saturation point. A guardrail checks its proposal before SIEM II reruns the same alerts, grouped and ranked. This document holds the background, architecture, formulas and the Codex task prompt.

The two terms

Term	In plain words
SOC (Security Operations Center)	The control room of the security team. Think of a mall's CCTV room: sensors raise alarms all day, and a few people decide which ones matter.
SRE (Site Reliability Engineering)	The engineers who keep an app like Shopee up and running. Their habit is to watch the traffic, set a limit for what the team can handle, and tune the alarms so on-call people are not woken up for noise.

The problem in one picture

Imagine a mall where the CCTV room gets 3,000 alarms a day and most are a door swinging open. The guards stop looking. One day a real break-in rings an alarm and nobody checks it. Surveys of security teams find that most alerts go unread, and many teams have had a breach tied to an alert that was ignored.

What our project does

We use Codex to do the SRE's job on the alarm system:

1. **Measure the flow:** how many alarms each rule raises per hour.
2. **Set a budget:** how many alarms the team can realistically read in an hour.
3. **Find the gap:** which rules raise far more than the budget allows, and which important ones raise too few.
4. **Change the rules:** group repeated alarms into one, and raise the priority when the same target is hit again and again. For example, two minor alarms on the same server become one medium alarm.
5. **Check the work:** a safety check makes sure no real attack is hidden and nothing harmless gets raised to high.

The result you can quote: in our test run, 857 separate alerts became 101 grouped ones, and an attack ranked Medium moved up to High. That test uses synthetic data. We will replace it with real figures before the demo.

Words you will hear

Term	In plain words
Suricata	The tool that watches network traffic and raises alerts.
PCAP	A recording of network traffic, like CCTV footage.
SIEM	The dashboard that collects all alerts. SIEM I uses the old rules, SIEM II the tuned rules.
Alert fatigue	When people ignore alarms because there are too many.

What a PM should look at

- **Alerts a human has to read:** fewer is better.
- **Real attacks ranked high:** this number must not drop.
- **Safety:** every rule change is a reviewable diff, and Codex cannot lower any score below its base.

Codex tuning as SRE: the delta gap calculation

Background and design notes for the Codex x Sea hackathon demo

Background: why alert tuning matters

Attacks are scaling up in Southeast Asia.

- An industry report found twice as many cyberattacks in Southeast Asia in 2024 as in 2023 [1].
- The hardest-hit sectors were banking and finance, retail, government and e-commerce [1]. That is the ground Shopee and Monee operate on.
- Kaspersky reported more than 18 million malicious detections against businesses in the region in 2025, with Vietnam, Malaysia and Indonesia the most affected [2].
- A report from Mafindo, based on 432,637 reports to the Indonesia Anti Scam Center, put online fraud losses in Indonesia at Rp 9.1 trillion [3].

Defenders cannot read what the sensors raise.

- Organizations average about 3,000 security alerts a day, and 63% of them go unaddressed (Vectra AI, 2026) [4].
- In a Prophet Security survey of nearly 300 security leaders, about 40% of alerts were never investigated, and 60% of teams had experienced breaches tied to ignored alerts [5].

So the bottleneck has moved from detection to attention. The sensors already see the attacks. The ignored alerts are where the breaches hide, which is the gap this project targets: group related alerts, rank them by risk, and let Codex tune the thresholds the way an SRE tunes capacity.

Pitch line: "Codex doesn't score alerts. It writes and tests the rules that do."

Note: these are vendor and survey figures. Cite the source on any slide. The 857-to-101 results in this document come from a synthetic sample and should be replaced with a run on the real eve_demo.json before the demo.

Design: the SRE framing

That fits the SRE framing well: you treat the alert stream like production traffic. The example numbers below come from running the built-in sample.

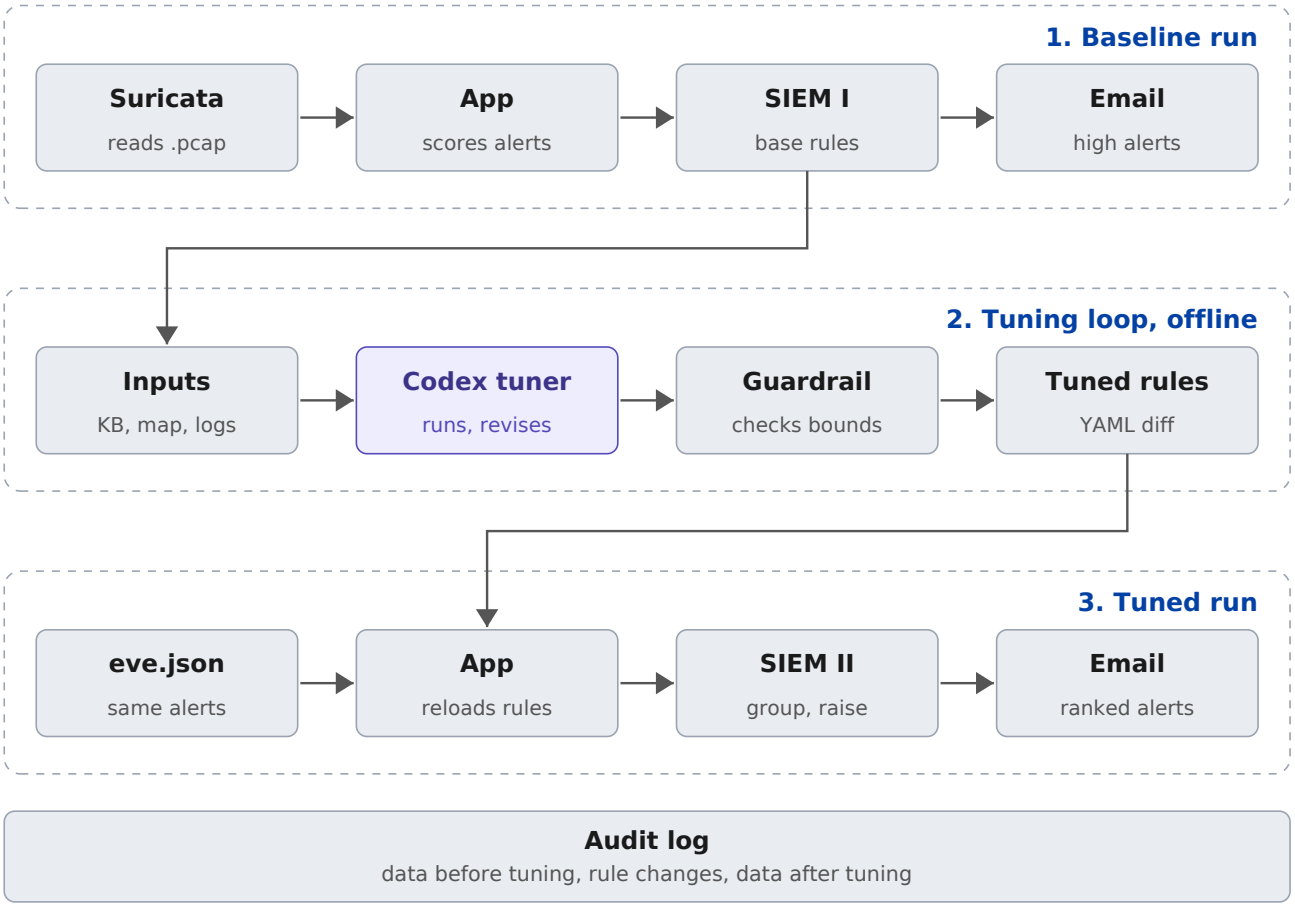
The framing: treat the alert stream as traffic, give the analysts a capacity budget, and have Codex compute where the rules put too much or too little load on it.

SRE concepts mapped to your SOC

SRE	Your system
Traffic	Alerts per hour per rule, from SIEM I
Capacity / SLO	An alert budget B, the alerts per hour analysts can handle
Burn rate	Observed alerts per hour divided by B
Priority tiers	Attack weight, service weight and attack-versus-benign evidence
Baseline and p95	Normal burst size n per rule
Rate-based alerting	The group window W and the saturation point N_sat

Architecture: three stages

The baseline run produces SIEM I's raw results. Codex works offline on those results and the rule KB, and a guardrail checks its proposal before the App reloads the tuned rules for the second run.



Purple marks the LLM step. Every other box is deterministic code.

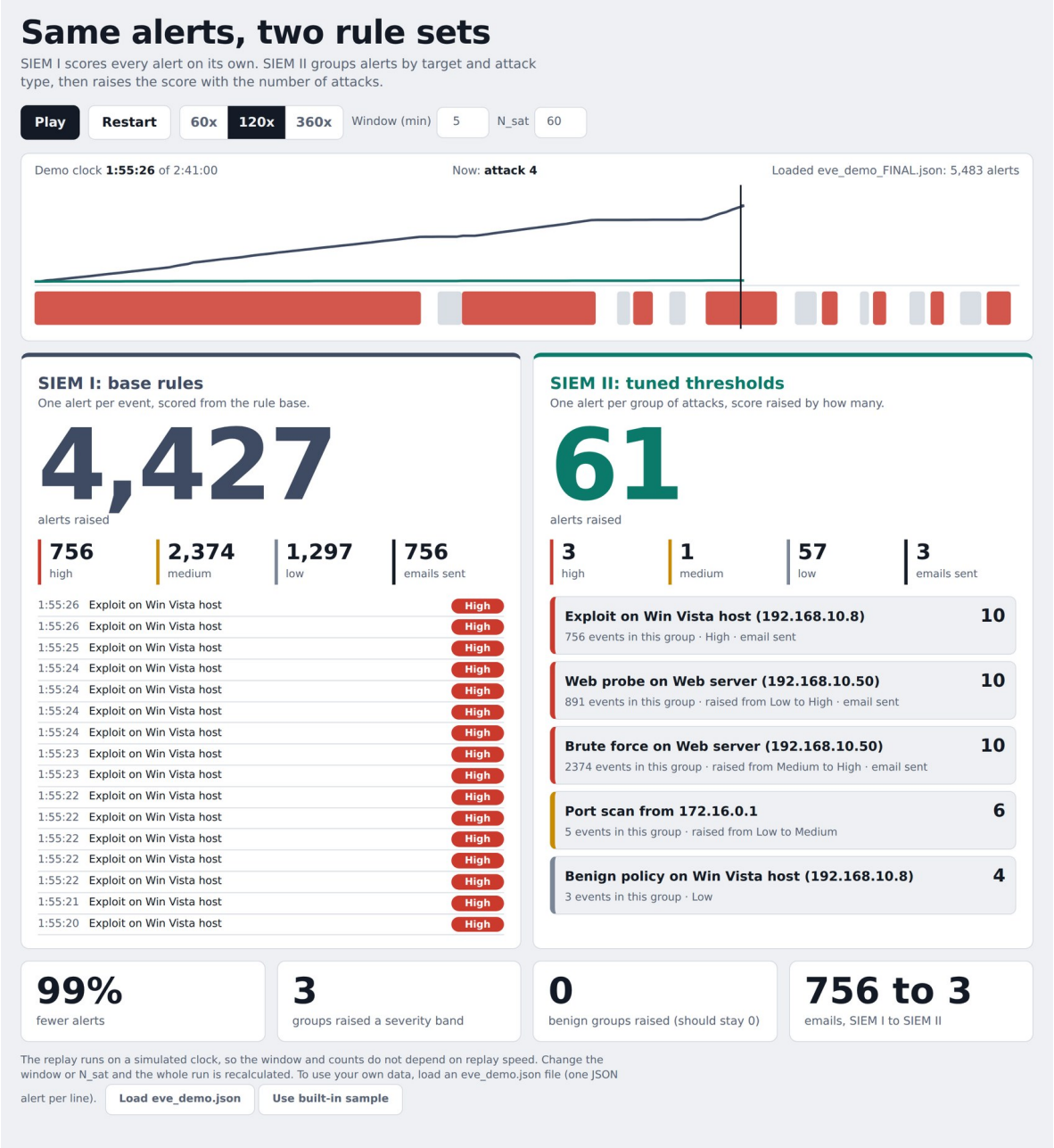
Runtime scoring in SIEM II (per event)

Step	Formula
Base score	S = min(cap, A + C)
Count	n = events in the same group in the last W minutes
Raised score	X = round(S + (cap − S) × min(1, ln n ÷ ln N_sat))
Group key	target + attack type; scanners use source + attack type
Bands	Low 1-4, Medium 5-7, High 8-10; benign alerts keep S

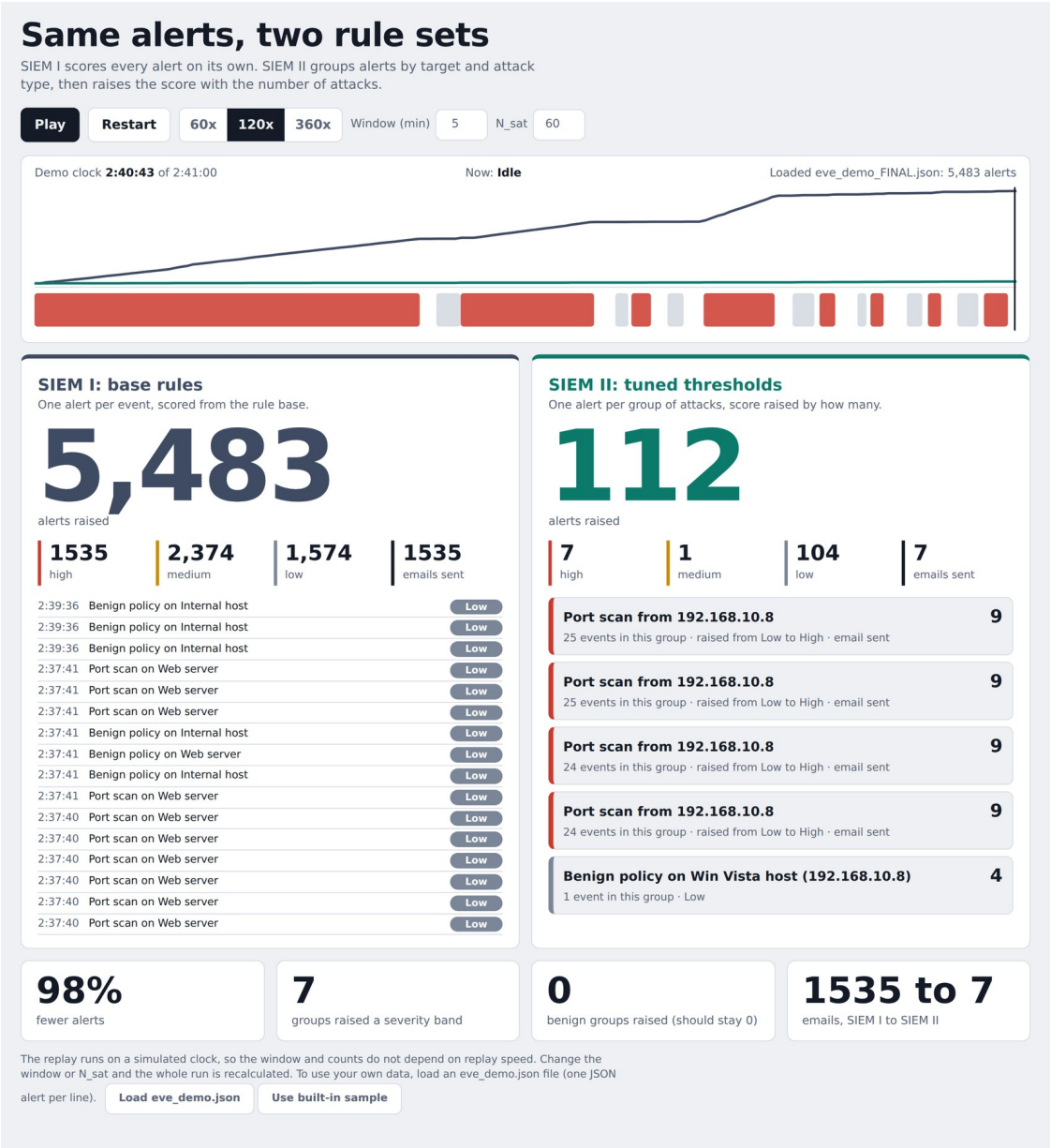
A is the attack-type weight, C the service weight from the IP mapping, and cap is 10. N_sat and W come from the Codex calculation in the next section.

Simulation prototype: the live replay page

The browser page that replays the alert log on a simulated clock, with SIEM I and SIEM II side by side



Mid-run, during the reverse-shell flood (demo clock 1:55:26 of 2:41:00, real eve_demo_FINAL.json, 5,483 alerts). SIEM I has raised 4,427 alerts, 756 of them High, so 756 emails. SIEM II has raised 61 grouped alerts and sent 3 emails. The exploit group on the Windows Vista host holds 756 events as a single alert, and no benign group has been raised.



End of the replay (demo clock 2:40:43). SIEM I: 5,483 alerts and 1,535 emails. SIEM II: 112 grouped alerts, 7 High, 7 emails, 98% fewer alerts, 7 groups raised a severity band, 0 benign groups raised.

How to read the page

- **Top strip:** cumulative alerts over time (dark line: SIEM I, green line: SIEM II) above the attack windows (red blocks).
- **Left lane, SIEM I:** every event is its own alert, scored from the rule base.
- **Right lane, SIEM II:** alerts grouped by target and attack type. Each card shows its events, its score and any band it was raised from.
- **Controls:** Play, Restart, speed (60x, 120x, 360x), and the window and N_sat boxes.
- **Important:** this page applies one global window (5 min) and one N_sat (60) and uses a host-only service map. It does not yet read the per-rule settings Codex writes to tuned_rules.yaml, which is why its SIEM II shows 112 alerts while the tuning script's evaluate command reports 46. Quote final numbers from evaluate until the page is updated.

The calculation Codex performs, per rule (attack type at a service)

1. **Flow:** volume per hour, peak n in the window, and p95 of n, all from SIEM I.
2. **Burn rate:** total alerts per hour divided by B.
3. **Priority weight:** w = S × evidence, where S is the base severity (A + C). Evidence is 1.0 for attack-labeled traffic and 0.2 for benign or unconfirmed traffic, which you would take from analyst feedback in production.
4. **Allowed share:** B × w / Σw.
5. **Delta gap:** observed alerts per hour − allowed share.
6. **Rule change:**
   - **Positive gap:** pick the smallest window W where grouped alerts per hour fit the allowed share. Set N_sat to the rule's p95 n, so a normal-peak burst reaches the cap and anything beyond is flagged.
   - **Negative gap:** the rule is under-alerting for its priority. Keep W small and raise sensitivity.

Real numbers from the sample (budget B = 20 per hour, W = 5)

The sample produces 373 alerts per hour, so the burn rate is 18.6x.

Rule	Observed per hour	Allowed per hour	Gap
Brute force at web server	194	3.2	+191
Benign policy at internal hosts	83	0.3	+83
Web probe at web server	51	2.6	+49
Benign policy at web server	25	0.4	+24
SQL injection at web server	9.1	4.5	+4.6
Port scan, internal	4.3	1.9	+2.4
Exploit at Windows Vista host	**3.9**	**5.8**	**−1.9**
Ping sweep, internal	2.2	1.3	+0.9

Codex should not tune every rule the same way. Brute force and benign noise need heavy grouping. The exploit rule has a negative gap, so it must keep alerting at once and never be merged.

Codex task prompt

```
You are an SRE-style tuning agent for a SIEM. Inputs: base_rules.yaml (attack
weights A, service weights C, current W and N_sat), siem1_results.json (one
record per alert), asset_map.yaml. Budget B = <alerts/hour> given by the operator.

1. Aggregate SIEM I output per rule (attack type x service): volume/hour, peak n
   in W, p95 n. Compute total burn rate = alerts/hour / B.
2. Compute each rule's priority weight w = S x evidence, its allowed share
   B*w/sum(w), and its delta gap = observed - allowed.
3. For positive gaps pick the smallest window W_i whose grouped alerts/hour fit
   the allowed share, and set N_sat_i = p95 n. For negative gaps keep W small and
   increase sensitivity. Never exceed a max W for rules with S >= 8.
4. Write tuned_rules.yaml with, for every change, old value, new value, the gap
   it addresses, and a one-sentence rationale.
5. Run evaluate(tuned_rules.yaml) and read back: alert reduction, attack-group
   recall, benign groups raised. If recall < 100% or any benign group is raised,
   revise and rerun (maximum 3 iterations). Report each iteration.
Constraints: never lower a score below its base S; do not edit anything except
tuned_rules.yaml; output only YAML plus a short report.
```

What the judges will see

Codex does real analysis (flow, budget, burn rate, gaps), it explains each change, and it tests its own proposal and revises it. The formula still executes the result, and the guardrail checks it.

Caveats to state honestly

- The budget B and the 0.2 evidence weight are assumptions. Say they are operator inputs, and that in production evidence would come from analyst feedback.
- The allowed-share model is simple. A rule with a very high weight could be allotted more alerts than it has, which is fine, since the gap then comes out negative.

Next step: a Python script that produces the per-rule flow table from eve_demo.json, plus the evaluate() function for Codex to call, so the loop is ready to run before the demo.

Design comparison

This project against TriageWall, a per-alert triage tool, and SIEM I against SIEM II inside this project

1. This project vs TriageWall

TriageWall is an open-source, local-LLM alert triage tool for self-hosted SOCs. The points below come from its public README [6], not from its code.

Aspect	TriageWall	This project
Problem	Triage each alert: repeat noise, or worth a look?	Alerts about one target are scored one by one, so floods bury attacks and slow attacks look harmless.
Input	Suricata eve.json, plus optional Wazuh.	Suricata eve.json from a PCAP replay.
Core method	A scoped rule filter for known repeat noise, then a local LLM for the leftover alerts.	A formula (base score, service weight, attack count, time window) that groups alerts and raises severity.
Role of the LLM	In the hot path: it judges residual alerts one by one, taking seconds per alert.	Offline tuner: Codex proposes thresholds from the rule KB and SIEM I results, then tests its own proposal.

Aspect	TriageWall	This project
Output	A review queue and analyst workbench.	Fewer, grouped, ranked alerts in SIEM II, plus email for the high ones.
Can it raise severity?	Not as described; it classifies and prioritizes.	Yes: repeated low alerts on one target are raised, for example two lows become one medium.
Config safety	Draft, preview, activate and roll back, with an audit trail.	Guardrail checks plus a YAML diff with a reason for every change.
Privacy	Everything stays local, no cloud model.	Codex is a cloud model; send it aggregated counts, not raw payloads, or note a local model can replace it.
Evidence	Evaluation on a labeled set of 266 alerts, reported as high accuracy and full recall.	Before/after alert counts, attack groups downgraded, benign groups raised.

Where the two overlap

Both read Suricata eve.json, use an LLM to cut alert noise, keep deterministic rules in charge, and measure results against a baseline.

Where this project differs

- **It correlates.** Alerts are grouped per target and attack type, and volume changes severity.
- **The LLM is off the hot path.** Codex runs once, offline, so there is no model call per alert and its output is a reviewable diff.
- **It can escalate,** not only suppress.

Ideas worth borrowing

- Report attack recall and benign false escalations next to the reduction number, using the attack/benign labels already in the data.
- Version the tuned rules with preview and rollback, and say plainly what the system does not do (it reads alerts and takes no response actions).

> **If a judge asks how this differs from TriageWall:** "TriageWall decides whether each alert is noise. We look across alerts: we group them per target and attack type and raise severity with volume, using thresholds Codex tunes offline."

2. SIEM I vs SIEM II (this project)

Aspect	SIEM I (baseline)	SIEM II (tuned)
Unit of alert	One alert per event	One alert per group of related events
Scoring	S = min(cap, A + C), each event on its own	Raised score: X = round(S + (cap - S) × min(1, ln n / ln N_sat))
Grouping	None	Same target and attack type; scanners grouped by source; window W per rule
Settings	One set of base rules	Per-rule window W and saturation N_sat, written by Codex and checked by a guardrail
Benign alerts	Scored like any other	Keep their base score; never raised
Email	One message per high alert	One message per high group

Sources

[1] Southeast Asia Infra, "Cyberthreats a catastrophe in the making" (citing an industry report and CloudSEK): southeastasiainfra.com/cyberthreats-a-catastrophe-in-the-making/ and southeastasiainfra.com/?p=51203

[2] Newswav, "Kaspersky reports over 18 million web attacks on SEA-based businesses in 2025": newswav.com (April 2026)

[3] The Jakarta Post, "Indonesia lost Rp 9.1 trillion to online fraud, report suggests", 28 Feb 2026: thejakartapost.com/indonesia/2026/02/28/indonesia-lost-rp-9-1-trillion-to-online-fraud-report-suggests

[4] Vectra AI, "Alert fatigue explained": vectra.ai/topics/alert-fatigue

[5] SecurityInfoWatch, "Prophet Security report highlights urgent need for AI in SOCs": securityinfowatch.com/ai/article/55315748/prophet-security-report-highlights-urgent-need-for-ai-in-socs

[6] TriageWall README (aaronphifer/triagewall): github.com/aaronphifer/triagewall